

Day 1 Project Setup & Patient Management Module

Date

2026-08-20

Overview

Today marks the foundation of the **Clinic Management System**. The goal was to initialize the Spring Boot project, establish MongoDB connectivity, implement the first core module (Patient Management), and create both REST API endpoints and a Thymeleaf-based web interface.

Technology Stack Used

Technology	Version	Purpose
Java	26	Programming Language
Spring Boot	4.1.0	Backend Framework
Spring Data MongoDB	4.1.0	Database Integration
MongoDB	Latest (local)	NoSQL Database
Thymeleaf	4.1.0	Server-side HTML templating
Bootstrap	5.3.2 (CDN)	UI Styling
Spring Validation	4.1.0	Request DTO validation
SpringDoc OpenAPI	2.8.0	API Documentation (prepared)
Lombok	1.18.46	Boilerplate code reduction
Maven	Bundled with IntelliJ	Build & Dependency Management
Git	Latest	Version Control

What Was Built Today

1. Project Initialization

- Created Maven-based Spring Boot project with `pom.xml`
- Configured `application.yml` with MongoDB connection string (`mongodb://localhost:27017/clinic_db`)
- Set server port to `8080`
- Enabled MongoDB auditing (`@EnableMongoAuditing`) for automatic `createdAt` and `updatedAt` timestamps

2. Package Structure Established

Created the complete layered architecture as per project requirements:

```
com.clinic/
├─ ClinicManagementSystemApplication.java
├─ config/                (reserved for future: SecurityConfig, OpenApiConfig)
├─ controller/            (REST + Thymeleaf controllers)
```

└─ dto/	(Request/Response Data Transfer Objects)
└─ entity/	(MongoDB Document entities)
└─ enums/	(Java enums for type safety)
└─ exception/	(reserved for future: custom exceptions)
└─ repository/	(Spring Data MongoDB repositories)
└─ security/	(reserved for future: JWT configuration)
└─ service/	(Business logic layer)
└─ mapper/	(reserved for future: entity-DTO mappers)
└─ util/	(reserved for future: utility classes)

3. Enums Created

Two enums were defined to ensure data consistency and avoid string literals in the database: `Gender.java`

- MALE
- FEMALE
- OTHER

`BloodGroup.java`

- A_POSITIVE, A_NEGATIVE
- B_POSITIVE, B_NEGATIVE
- AB_POSITIVE, AB_NEGATIVE
- O_POSITIVE, O_NEGATIVE

4. Patient Entity (`Patient.java`)

Created the core `Patient` document mapped to MongoDB collection `patients`:

Field	Type	Description
patientId	String	Custom ID (e.g., PAT-A1B2C3D4)
name	String	Patient full name
email	String	Contact email
phone	String	Contact phone
gender	Gender	Enum: MALE/FEMALE/OTHER
dateOfBirth	LocalDate	Date of birth
address	String	Residential address
bloodGroup	BloodGroup	Enum: A_POSITIVE, O_NEGATIVE, etc.
createdAt	LocalDateTime	Auto-populated by @CreateDate
updatedAt	LocalDateTime	Auto-populated by @LastModifiedDate

Annotations used: @Document, @Id, @CreateDate, @LastModifiedDate, @Data, @Builder, @NoArgsConstructor, @AllArgsConstructor

5. DTOs (Data Transfer Objects)

`PatientRequestDto.java` Used for incoming API requests:

- Validation annotations applied:

- `@NotBlank` on name, email, phone
- `@Email` on email
- `@Size(min=10, max=15)` on phone
- `@NotNull` on gender, dateOfBirth, bloodGroup
- Lombok annotations: `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`

PatientResponseDto.java Used for outgoing API responses:

- Mirrors entity fields including `patientId`, `createdAt`, `updatedAt`
- Separates internal entity from external contract
- Lombok annotations: `@Data`, `@Builder`, `@NoArgsConstructor`, `@AllArgsConstructor`

6. Repository Layer (**PatientRepository.java**)

Extends `MongoRepository<Patient, String>` with custom query methods:

- `findByEmail(String email)` For duplicate checking (future use)
- `findByPhone(String phone)` For duplicate checking (future use)
- `findByNameContainingIgnoreCase(String name)` For search functionality

7. Service Layer (**PatientService.java** + **PatientServiceImpl.java**)

Interface: Defines the contract for patient operations.

Implementation: Contains business logic:

- `createPatient()` Generates custom `PAT-XXXX` ID using `UUID`, saves to MongoDB
- `getAllPatients()` Retrieves all patient documents
- `getPatientById()` Finds by `patientId`, throws `RuntimeException` if not found (will be replaced with custom exception on Day 2)
- `updatePatient()` Full update of all fields
- `deletePatient()` Deletes by ID after verifying existence
- `searchPatientsByName()` Case-insensitive name search
- `mapToResponseDto()` Private helper method to convert `Patient` entity → `PatientResponseDto`

Design patterns:

- Constructor-based dependency injection via `@RequiredArgsConstructor`
- Builder pattern via `@Builder` for object creation

8. REST API Controller (**PatientRestController.java**)

Base path: `/api/patients`

Method	Endpoint	Description	Status Codes
POST	<code>/api/patients</code>	Create new patient	201 Created

GET	/api/patients	List all patients	200 OK
GET	/api/patients/{id}	Get patient by ID	200 OK
PUT	/api/patients/{id}	Update patient	200 OK
DELETE	/api/patients/{id}	Delete patient	204 No Content
GET	/api/patients/search? name=	Search by name	200 OK

Features:

- `@Valid` annotation triggers DTO validation
- `ResponseEntity` used for proper HTTP status codes
- `@RequestBody` for JSON input, `@PathVariable` for IDs, `@RequestParam` for search
- `@RequiredArgsConstructor` for dependency injection

9. Thymeleaf Web Controllers

`IndexController.java` Serves the homepage at /

`PatientViewController.java` Serves patient list at /patients

- Injects `PatientService` to fetch all patients
- Adds `patients` attribute to Spring `Model`
- Returns view name `patients/list`
- Uses `@RequiredArgsConstructor` for injection

10. Thymeleaf HTML Templates

`templates/index.html` Landing page

- Bootstrap 5 navbar with navigation links
- Welcome message and link to patient list
- Responsive container layout `templates/patients/list.html` Patient listing page
- Bootstrap 5 styled table with dark header
- Thymeleaf `th:each` iteration over patient list
- Displays all 9 patient fields in columns
- Empty state message when no patients exist
- Badge showing total patient count
- Alert box listing all available REST endpoints for reference

Bootstrap integration: Via CDN (no local CSS files yet):

- **CSS:** <https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/css/bootstrap.min.css>
 - **JS:** <https://cdn.jsdelivr.net/npm/bootstrap@5.3.2/dist/js/bootstrap.bundle.min.js>
-

Testing Performed

REST API Tests (via Postman / Browser)

1. Create Patient

```
POST http://localhost:8080/api/patients
{
  "name": "Ali Khan",
  "email": "ali@example.com",
  "phone": "03001234567",
  "gender": "MALE",
  "dateOfBirth": "1990-05-15",
  "address": "123 Main Street, Karachi",
  "bloodGroup": "O_POSITIVE"
}
```

- Result: 201 Created with auto-generated patientId: "PAT-A1B2C3D4"
- createdAt and updatedAt auto-populated

2. List Patients

- GET http://localhost:8080/api/patients
- Result: Array containing the created patient

3. Get By ID

- GET http://localhost:8080/api/patients/PAT-A1B2C3D4
- Result: Single patient object

4. Update Patient

- PUT http://localhost:8080/api/patients/PAT-A1B2C3D4
- Result: Updated patient with new updatedAt timestamp

5. Search

- GET http://localhost:8080/api/patients/search?name=Ali
- Result: Filtered list matching "Ali" (case-insensitive)

6. Delete

- DELETE http://localhost:8080/api/patients/PAT-A1B2C3D4
- Result: 204 No Content

Thymeleaf UI Tests

- http://localhost:8080/ → Homepage renders correctly
- http://localhost:8080/patients → Table renders with patient data after REST creation
- Empty state displays correctly when database is clean

Database Verification

- Opened MongoDB Compass
- Connected to mongodb://localhost:27017
- Database clinic_db created automatically

- `Collection patients` contains documents with custom `patientId` as `_id`
- `createdAt` and `updatedAt` fields present as `ISODate`

Challenges Faced & Solutions

Challenge	Solution
Java 26 compiler incompatibility with Lombok	Upgraded Lombok to 1.18.46 and Spring Boot to 4.1.0
Java 26 <code>TypeTag :: UNKNOWN</code> compiler crash with Spring Boot 3.4.1	Created a separate test project to verify Lombok 1.18.46 + Java 26 compatibility, then upgraded main project to Spring Boot 4.1.0
ThemeSource NoClassDefFoundError on startup	Cleaned Maven cache and reloaded dependencies
Incompatible class format / Unsupported class file major version 70	Upgraded Spring Boot from 3.4.1 to 4.1.0 to gain Java 26 support
mvn command not recognized in PowerShell	Used IntelliJ built-in Maven panel (Lifecycle → clean → install) instead of terminal
Git push failed (src refspec main does not match any)	Committed locally first (<code>git commit</code>) before pushing to remote
Old test data with MongoDB ObjectIds instead of custom PAT-XXXX	Cleared collection and recreated patients via current REST API

Git Activity

- **Repository:** <https://github.com/abdul15x/clinic-management-system>
- **Branch:** `main`
- **Commit:** `feat: day 1 complete - patient CRUD with Lombok, Java 26, Spring Boot 4.1.0`
- **Files pushed:** All source code, `pom.xml`, `application.yml`, HTML templates

Files Created Today (Complete List)

```
pom.xml
src/main/resources/application.yml
src/main/resources/templates/index.html
src/main/resources/templates/patients/list.html
src/main/java/com/clinic/ClinicManagementSystemApplication.java
src/main/java/com/clinic/enums/Gender.java
src/main/java/com/clinic/enums/BloodGroup.java
src/main/java/com/clinic/entity/Patient.java
src/main/java/com/clinic/dto/PatientRequestDto.java
src/main/java/com/clinic/dto/PatientResponseDto.java
src/main/java/com/clinic/repository/PatientRepository.java
src/main/java/com/clinic/service/PatientService.java
```

```
src/main/java/com/clinic/service/PatientServiceImpl.java
src/main/java/com/clinic/controller/PatientRestController.java
src/main/java/com/clinic/controller/PatientViewController.java
src/main/java/com/clinic/controller/IndexController.java
```

What's Next (Day 2 Preview)

- Add custom exceptions (`PatientNotFoundException`, `DuplicateEmailException`)
 - Replace `RuntimeException` with proper exception handling
 - Implement pagination: `GET /api/patients?page=0&size=10`
 - Add Thymeleaf patient creation form (`patients/form.html`)
 - Add Thymeleaf patient detail view (`patients/detail.html`)
 - Add `@RestControllerAdvice` global exception handler (Day 11 target, but basic setup on Day 2)
-

How to Run This Version

1. Ensure MongoDB is running on `localhost:27017`
2. Open project in IntelliJ IDEA
3. Set Project SDK to your installed JDK (Java 26)
4. Set Language Level to 26
5. Enable Annotation Processing: `Settings → Build, Execution, Deployment → Compiler → Annotation Processors → Check "Enable annotation processing"`
6. Run `ClinicManagementSystemApplication.java` (green play button)
7. Access:
 - Web UI: `http://localhost:8080/`
 - Patient List: `http://localhost:8080/patients`
 - REST API: `http://localhost:8080/api/patients`